SOKOINE UNIVERSITY OF AGRICULTURE

School of Engineering and Technology
Department of Agricultural Engineering

AGE 219: Basics of Computer Programming

Lecture Compendium 2: Variables, Data Types, and Basic Operators in Engineering Computation

---

Prepared by:

Dr. Kadeghe Fue, PhD, P.Eng (T)

Target Audience: Engineering Students (Agricultural, Irrigation, Post-Harvest, and related fields)
Primary Tools: Python 3.x and Google Colaboratory
Location: Morogoro, Tanzania

This document serves as the primary study material for Module 2 of AGE 219. It transitions from basic software execution to the fundamental building blocks of digital agriculture: storing, identifying, and manipulating physical farm data.

Contents

1 Introduction: The Anatomy of Digital Data **2**

2 Deep Dive into Variables **2**
2.1 Dynamic Typing in Python . . . 2
2.2 Variable Naming Conventions (PEP 8) . . . 2

3 The Four Primitive Data Types **3**
3.1 1. Integers (`int`) . . . 3
3.2 2. Floating-Point Numbers (`float`) . . . 3
3.3 3. Strings (`str`) . . . 4
3.4 4. Booleans (`bool`) . . . 4

4 Type Checking and Casting (Data Conversion) **4**
4.1 Checking the Type . . . 4
4.2 Type Casting . . . 4

5 Operators: The Tools of Computation **5**
5.1 1. Arithmetic Operators . . . 5
5.2 2. Assignment Operators . . . 6
5.3 3. Comparison (Relational) Operators . . . 6

6 Case Studies in Agricultural Engineering **7**
6.1 Case Study 1: Precision Fertilizer Application Rate . . . 7
6.2 Case Study 2: Harvester Logistics and Fuel Range . . . 7

7 Debugging Common Type Errors **8**
7.1 1. TypeError . . . 8
7.2 2. ValueError . . . 9

8 Summary **9**

9 Glossary of Terms **10**

10 Extended Self-Assessment Exercises **11**
10.1 Exercise 1: The Sensor Calibration Script . . . 11
10.2 Exercise 2: Grain Storage Geometry . . . 11
10.3 Exercise 3: Boolean Logic Drill . . . 11
10.4 Exercise 4: Tractor Service Interval (Modulo Practice) . . . 11

1 Introduction: The Anatomy of Digital Data

In Lecture 1, we learned that a computer program fundamentally follows the IPO model: **Input**, **Process**, **Output**. We wrote a simple script to calculate the volume of a grain silo. However, agricultural systems are rarely static. A smart irrigation system does not just calculate a value once; it continuously reads data from soil sensors, processes that data against changing weather forecasts, and outputs commands to valves.

To manage this continuous flow of information, the computer must have a way to temporarily store, classify, and manipulate the data in its Random Access Memory (RAM). In this lecture, we will explore the core tools for this task: **Variables** (the storage containers), **Data Types** (the classification system), and **Operators** (the mathematical and logical manipulation tools).

2 Deep Dive into Variables

A variable is a named location in the computer's memory used to store data. Think of the computer's RAM as a massive warehouse full of thousands of blank, empty boxes.

When you create a variable in Python, you are doing three things:

1. Grabbing an empty box in the warehouse.
2. Writing a label on the outside of the box (the Variable Name).
3. Placing a value inside the box (the Data).

2.1 Dynamic Typing in Python

In older, traditional engineering languages like C or C++ (which are often used to program microcontrollers like Arduino), you must tell the computer exactly what *kind* of data is going into the box before you put it there. This is called **Static Typing**.

Python, however, uses **Dynamic Typing**. You do not need to declare the type in advance. Python is intelligent enough to examine the value you are putting into the box and adjust the box's size and properties automatically.

```
# In C++ (Static Typing - For context only, DO NOT RUN)
# float engine_temperature = 85.5;

# In Python (Dynamic Typing - Run in Colab)
engine_temperature = 85.5
```

2.2 Variable Naming Conventions (PEP 8)

While Python gives you the freedom to name variables almost anything, the global programming community follows a strict style guide called PEP 8. For agricultural engineers, writing readable code is a matter of safety and efficiency. If your code controls a 10-ton autonomous tractor, another engineer must be able to read and understand your variables instantly.

The Rules of Naming:

- **Allowed Characters:** Only letters (a-z, A-Z), numbers (0-9), and underscores (_).
- **No Starting Numbers:** A variable cannot start with a number (e.g., `1st_tractor` is illegal; `tractor_1` is correct).
- **Case Sensitivity:** `Yield` and `yield` are completely different boxes in memory.

- **Reserved Keywords:** You cannot use Python's built-in commands as variable names (e.g., you cannot name a variable `print`, `if`, or `for`).

The Engineering Standard (Snake Case): Always use lowercase words separated by underscores. Make the name descriptive of the physical quantity it represents, and ideally, include the unit of measurement.

```
# POOR naming convention:
t = 25
w = 1200
v1 = 3.5

# EXCELLENT engineering naming convention (Snake Case):
ambient_temp_celsius = 25
tank_capacity_liters = 1200
tractor_speed_kmh = 3.5
```

3 The Four Primitive Data Types

Just as physical materials in an engineering workshop have different properties (steel is rigid, rubber is flexible, oil is viscous), digital data has different properties. The type of data dictates what operations you can perform on it. You can multiply two numbers, but you cannot mathematically divide a word.

Python has four primary "Primitive" data types.

3.1 1. Integers (`int`)

Integers are whole numbers without a decimal point. They can be positive, negative, or zero.

- **Agricultural Context:** Integers are used for counting discrete, indivisible items. You cannot have 2.5 tractors, or 10.7 irrigation zones.
- **Memory:** Integers require very little RAM and processing power, making them highly efficient for the CPU.

```
# Examples of Integers
number_of_tractors = 5
total_irrigation_valves = 24
days_since_planting = -2 # Indicating 2 days before planting
```

3.2 2. Floating-Point Numbers (`float`)

Floats are numbers that contain a decimal point.

- **Agricultural Context:** Floats are the most common data type in engineering. They represent continuous physical measurements: temperature, weight, volume, speed, and GPS coordinates.
- **Engineering Warning:** Because computers use binary (base-2) math, floating-point numbers sometimes have microscopic precision errors (e.g., $0.1+0.2$ might equal 0.30000000000000004 in a computer). For most agricultural applications, this microscopic difference is irrelevant, but it is important to know for high-precision financial or chemical calculations.

```
# Examples of Floats
soil_ph_level = 6.5
pump_flow_rate_lps = 15.75
gps_latitude = -6.8488 # Latitude of SUA, Morogoro
```

3.3 3. Strings (`str`)

Strings are sequences of text characters. They must be enclosed in either single quotes (`'`) or double quotes (`"`).

- **Agricultural Context:** Strings are used for labels, farm block names, machinery serial numbers, and displaying messages to the farmer.
- **String Concatenation:** You can "add" strings together using the + operator, which joins them end-to-end.

```
# Examples of Strings
tractor_model = "John Deere 5075E"
field_block = 'Block C'

# String Concatenation
status_message = "Currently harvesting " + field_block
print(status_message) # Outputs: Currently harvesting Block C
```

3.4 4. Booleans (`bool`)

Booleans are the simplest data type. They can only hold one of two values: `True` or `False` (Note the capital T and F in Python).

- **Agricultural Context:** Booleans represent binary states. They are the digital equivalent of an electrical switch being ON or OFF. They are essential for control systems and decision-making logic.

```
# Examples of Booleans
is_raining = False
water_tank_full = True
pto_engaged = False
```

4 Type Checking and Casting (Data Conversion)

When reading data from external sensors or user inputs, the computer may not interpret the data type correctly. For instance, a temperature sensor might send the reading `"25.5"` as a String of text, rather than a Float number. If you try to do math on a String, your program will crash.

4.1 Checking the Type

You can ask Python what type of data is currently stored in a variable using the `type()` function.

```
sensor_reading = 45.2
print(type(sensor_reading))
# Output: <class 'float'>
```

4.2 Type Casting

Casting is the process of forcing a variable to change its data type. We use the built-in functions `int()`, `float()`, and `str()` to convert data.

```python
# Scenario: A sensor sends data as a string
raw_sensor_data = "150"
# print(raw_sensor_data + 10) -> THIS WOULD CAUSE AN ERROR

# Casting the string to an integer
processed_data = int(raw_sensor_data)

# Now math is possible
new_total = processed_data + 10
print(new_total) # Output: 160
```

The `input()` Function Trap

When you use the `input()` function to ask a user to type something into the Colab notebook, **Python always reads the input as a String**, even if the user types a number. You must cast it to a `float` or `int` before doing engineering calculations.

```python
# Correct way to handle user input for math
user_input = input("Enter the farm area in Hectares: ")
farm_area_ha = float(user_input) # Casts the text to a decimal number
area_sq_meters = farm_area_ha * 10000
```

5 Operators: The Tools of Computation

Operators are special symbols in Python that carry out arithmetic or logical computation. The value that the operator operates on is called the *operand*.

5.1 1. Arithmetic Operators

These are used to perform common mathematical operations.

- **Addition (+):** Adds values on either side of the operator.
- **Subtraction (-):** Subtracts right hand operand from left hand operand.
- **Multiplication (*):** Multiplies values on either side of the operator.
- **Division (/):** Divides left hand operand by right hand operand. *Note: Division in Python 3 always results in a Float, even if the numbers divide perfectly.*

Advanced Engineering Operators:

- **Floor Division (//):** The division of operands where the result is the quotient in which the digits after the decimal point are removed.
- **Modulo (%):** Divides left hand operand by right hand operand and returns the **remainder**. This is incredibly useful for converting units (like total minutes into hours and minutes) or determining if a row pattern is even or odd.
- **Exponentiation (**):** Performs exponential (power) calculation on operators. Equivalent to x^y.

```python
# Advanced Arithmetic Examples
total_minutes_pumping = 145

# Floor division to get whole hours
hours = total_minutes_pumping // 60  # Result is 2

# Modulo to get the remaining minutes
remaining_minutes = total_minutes_pumping % 60 # Result is 25

print("The pump ran for", hours, "hours and", remaining_minutes, "minutes.")
```

5.2 2. Assignment Operators

We have already used the basic assignment operator (=) to assign values to variables. Python also offers "compound" assignment operators that perform math and assignment in a single step.

- `x += 5` is exactly the same as `x = x + 5`
- `x -= 2` is exactly the same as `x = x - 2`
- `x *= 10` is exactly the same as `x = x * 10`

```python
# Updating a water tank volume
current_volume_L = 500
inflow_L = 50

# The tank receives 50 liters. Update the variable:
current_volume_L += inflow_L
print(current_volume_L) # Output: 550
```

5.3 3. Comparison (Relational) Operators

These operators compare the values on either side of them and decide the relation among them. The result of a comparison operator is **always a Boolean** (`True` or `False`). These are the foundation of decision-making algorithms (which we will cover in Lecture 3).

- **Equal to (==):** Notice the DOUBLE equals sign. A single = assigns a value. A double == asks a question: "Are these two equal?"
- **Not Equal to (!=):** Returns True if values are different.
- **Greater than (>):**
- **Less than (<):**
- **Greater than or equal to (>=):**
- **Less than or equal to (<=):**

```python
# Sensor comparison logic
current_moisture_percent = 28.5
moisture_threshold = 30.0

# Is the current moisture strictly less than the threshold?
needs_water = current_moisture_percent < moisture_threshold

print("Does the field need water?")
print(needs_water) # Output: True
```

6 Case Studies in Agricultural Engineering

Let us integrate variables, data types, casting, and operators into practical engineering workflows. Open Google Colab and type out these case studies.

6.1 Case Study 1: Precision Fertilizer Application Rate

Background: An agronomist has recommended a Nitrogen application rate of 120 kg/ha for a specific maize field. The farmer's field is rectangular, measured in meters. The fertilizer available is Urea, which contains 46% Nitrogen by weight.

Engineering Problem: Write a script that takes the field dimensions as inputs, calculates the exact total amount of Urea (in kg) required for the field, and determines how many standard 50kg bags the farmer needs to purchase.

Algorithmic Logic (IPO): 1. **Input:** Length (m), Width (m), Nitrogen requirement (kg/ha), Urea Nitrogen %. 2. **Process:** - Calculate area in m^2, then convert to Hectares. - Calculate total pure Nitrogen needed for the whole farm. - Calculate total Urea needed (since Urea is only 46% N). - Divide total Urea by 50kg to get the number of bags. Use *floor division* and *modulo* to understand partial bags. 3. **Output:** Display the results clearly using string concatenation.

Python Implementation:

```
# --- 1. Inputs ---
# (In a real system, these might come from user input() functions)
field_length_m = 450.0
field_width_m = 300.0
target_n_rate_kgha = 120.0
urea_n_percentage = 0.46
bag_weight_kg = 50

# --- 2. Processing ---
# Area calculation
area_m2 = field_length_m * field_width_m
area_ha = area_m2 / 10000.0

# Nutrient calculation
total_n_required_kg = area_ha * target_n_rate_kgha
total_urea_required_kg = total_n_required_kg / urea_n_percentage

# Logistics calculation (How many whole bags?)
# We use integer casting and floor division
whole_bags_needed = int(total_urea_required_kg // bag_weight_kg)
remaining_kg = total_urea_required_kg % bag_weight_kg

# --- 3. Outputs ---
# We use f-strings (formatted strings) for cleaner output printing
# Placing an 'f' before the string allows us to put variables in curly braces {}
print(f"Total Farm Area: {area_ha} Hectares")
print(f"Total Urea Required: {total_urea_required_kg} kg")
print(f"Purchase Required: {whole_bags_needed} whole 50kg bags, plus an additional {
    remaining_kg} kg.")
```

Note on f-strings: In the code above, we introduced **f-strings** (e.g., `f"Area: {area_ha}"`). This is the modern, cleanest way in Python to mix text (Strings) and numbers (Floats/Ints) in a single `print()` statement without needing to cast everything to strings manually.

6.2 Case Study 2: Harvester Logistics and Fuel Range

Background: A combine harvester is operating in an expansive wheat field in Hanang. The onboard telematics computer needs to calculate if the harvester has enough fuel to complete

the current block without stopping.

Given Parameters:

- Remaining Fuel: 185 Liters
- Fuel Consumption Rate: 32 Liters per Hour
- Harvester Speed: 6.5 km/hr
- Implement Width: 7.5 meters
- Field Block Size to finish: 25 Hectares

Python Implementation:

```
# Define variables
fuel_remaining_L = 185.0
consumption_rate_L_hr = 32.0
speed_kmh = 6.5
width_m = 7.5
target_area_ha = 25.0

# 1. Calculate Theoretical Field Capacity (TFC in ha/hr)
# Formula: TFC = (Speed * Width) / 10
tfc_ha_hr = (speed_kmh * width_m) / 10.0

# 2. Calculate time required to finish the block
time_required_hr = target_area_ha / tfc_ha_hr

# 3. Calculate total fuel required for that time
fuel_required_L = time_required_hr * consumption_rate_L_hr

# 4. Logical Decision: Do we have enough fuel?
# This will output a Boolean (True or False)
can_finish_block = fuel_remaining_L >= fuel_required_L

# Outputs
print(f"Time required to harvest {target_area_ha} ha: {time_required_hr} hours.")
print(f"Fuel required: {fuel_required_L} Liters.")
print(f"Sufficient fuel onboard? {can_finish_block}")
```

7 Debugging Common Type Errors

As you write code dealing with different data types, you will inevitably encounter errors. Understanding these specific errors is crucial for rapid debugging.

7.1 1. TypeError

A `TypeError` occurs when you attempt an operation that is invalid for the specific data type. The most common engineering mistake is trying to perform math on a String.

```
sensor_data = "45"
offset = 10
# The following line will crash the program:
result = sensor_data + offset
```

Colab Error Output: `TypeError: can only concatenate str (not "int") to str`
Solution: You must cast the string to an integer first: `int(sensor_data) + offset`.

7.2 2. ValueError

A `ValueError` occurs when a function receives an argument of the correct type, but an inappropriate value. For example, trying to cast a word into a number.

```
user_input = "Five"
# The following line will crash:
number = int(user_input)
```

Colab Error Output: `ValueError: invalid literal for int() with base 10: 'Five'`
Solution: The `int()` function can only convert strings that actually contain digits (e.g., `"5"`). It cannot understand English words.

8 Summary

In this lecture, we have laid the foundation for processing physical farm data in a digital environment. You have learned:

- How **Dynamic Typing** makes Python highly efficient for scripting, and how to follow **PEP 8** naming conventions.
- The four primitive data types: **Integers** (counts), **Floats** (continuous measurements), **Strings** (text), and **Booleans** (binary states).
- How to use **Casting** functions (`int()`, `float()`, `str()`) to ensure data is in the correct format for mathematical operations.
- How to utilize **Arithmetic, Assignment, and Comparison Operators** to manipulate data and generate boolean logic states.
- How to apply these concepts to real-world engineering problems like fertilizer calculation and machinery logistics.

In Lecture 3, we will take the Boolean outputs generated by our Comparison Operators and use them to build **Control Flow (Decision Making)**, allowing our programs to automatically react to changing farm conditions.

9 Glossary of Terms

- **Boolean:** A data type that has one of two possible values (usually denoted True and False), intended to represent the two truth values of logic and Boolean algebra.
- **Casting:** In programming, type casting is a method used to change the variables/values from one data type to another data type.
- **Concatenation:** The operation of joining two strings of characters end-to-end.
- **Dynamic Typing:** A programming language paradigm where the type of a variable is checked and determined during runtime (when the program is running), rather than during compilation. Python is dynamically typed.
- **Float:** A data type used to represent real numbers that contain a fractional part (a decimal point).
- **Integer:** A data type representing whole numbers, without fractional or decimal components.
- **Modulo:** An arithmetic operation that finds the remainder after division of one number by another.
- **Operand:** The object of a mathematical operation; a quantity on which an operation is performed.
- **Operator:** A character or characters that determine the action that is to be performed or considered (e.g., +, ==).
- **PEP 8:** Python Enhancement Proposal 8; the official style guide for Python code, detailing how to format code and name variables for maximum readability.
- **String:** A sequence of characters, either as a literal constant or as some kind of variable, representing text.

10 Extended Self-Assessment Exercises

Open a new Google Colab notebook specifically for Lecture 2 and complete the following algorithmic exercises. Ensure all variable names follow PEP 8 Snake Case formatting.

10.1 Exercise 1: The Sensor Calibration Script

A soil moisture sensor returns a raw voltage reading between 0.0V and 5.0V. You need to convert this voltage into a volumetric water content percentage (0% to 100%).

- Assume the relationship is linear.
- 0.0V = 0% moisture.
- 5.0V = 100% moisture.

Task: Write a script that declares a variable `raw_voltage` (set it to 3.2), calculates the corresponding moisture percentage, and prints the result using an f-string.

10.2 Exercise 2: Grain Storage Geometry

You are designing a rectangular grain bunker. **Task:** Use the `input()` function to ask the user to input the Length, Width, and Height of the bunker in meters. *(Remember the `input()` trap! You must cast these inputs to floats).* Calculate the total volume of the bunker in cubic meters. If maize has a bulk density of 720 kg/m^3, calculate the total mass of maize the bunker can hold in Metric Tons (1 Metric Ton = 1000 kg). Print the final tonnage.

10.3 Exercise 3: Boolean Logic Drill

Without running the code, determine what the output (True or False) of each of these print statements will be. Write your answers on a piece of paper, then type the code into Colab to check if you were correct.

```
flow_rate = 120.5
max_capacity = 150.0
valve_open = True

print(flow_rate > 100)
print(flow_rate == max_capacity)
print(valve_open != False)
print(max_capacity <= 150.0)
```

10.4 Exercise 4: Tractor Service Interval (Modulo Practice)

A tractor requires a minor engine service every 250 operating hours. **Task:** The tractor's dashboard currently reads exactly 1135 total operating hours. Write a script using the Modulo (%) and Floor Division (//) operators to calculate and print two things: 1. How many total minor services the tractor has completed in its lifetime. 2. How many hours have passed since the last minor service.

End of Lecture 2 Compendium